

Agile

1. Introduction to Agile

What is Agile?

Agile is a set of values, principles and practices for developing software (and other products) incrementally and iteratively. It emphasizes collaboration, customer feedback, responding to change, and delivering working increments frequently.

Core goals of Agile

- Deliver valuable working product frequently (short cycles).
- Embrace changing requirements, even late in development.
- Reduce risk by iterative feedback and continuous delivery.
- Improve collaboration between cross-functional teams and stakeholders.
- Increase predictability and transparency.

Agile vs Waterfall — Comparison

Waterfall (traditional)

- Linear, sequential phases (requirements → design → build → test → deploy).
- Little room for change once a phase is complete.
- Longer feedback loops; working product often delivered late.
- High risk of mismatch between requirements and final product.

Agile

- Iterative and incremental (short timeboxed iterations/sprints).

- Frequent feedback and adaptation; changes expected.
- Working product increments delivered regularly.
- Collaboration-focused; cross-functional teams.

When each is appropriate

- Waterfall: stable requirements, regulatory-heavy projects with strict phases.
 - Agile: evolving requirements, need for fast feedback, customer-driven products.
-

2. The Agile Manifesto

The four values (in plain words)

1. Individuals and interactions over processes and tools.

Tools and processes matter, but communication and people are more important.

2. Working software over comprehensive documentation.

Documentation useful, but priority is delivering working product.

3. Customer collaboration over contract negotiation.

Engage customers and stakeholders continuously rather than rely only on fixed contracts.

4. Responding to change over following a plan.

Plans are important but should not block adaptation to better ideas or information.

Note: The items on the right are still important — Agile values prioritize the left items when there's a conflict.

The twelve principles (high-level summary)

1. Satisfy the customer through early and continuous delivery.
2. Welcome changing requirements, even late in development.

3. Deliver working software frequently (weeks rather than months).
4. Business people and developers must work together daily.
5. Build projects around motivated individuals; give them support and trust.
6. Face-to-face conversation (or close collaboration) is the most efficient method of communication.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development; sponsors, developers and users should maintain a constant pace.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing work not done—is essential.
11. The best architectures, requirements and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects and adjusts behavior to become more effective.

How values/principles guide practices

- Short iterations and frequent demos come from “early and continuous delivery” and “working software as progress”.
 - Daily standups are driven by “communication and collaboration”.
 - Retrospectives follow the principle of continuous improvement.
 - Backlog refinement & flexible planning follow “welcome changing requirements”.
-

3. Scrum Framework

Scrum is a widely used Agile framework for managing product development. It prescribes roles, events (ceremonies), and artifacts.

3.1 Scrum roles

Product Owner (PO)

- Responsible for maximizing product value.
- Owns and prioritizes the Product Backlog.
- Interfaces with stakeholders, customers and the development team.
- Accepts or rejects increment at Sprint Review/acceptance criteria.

Responsibilities:

- Define user stories and acceptance criteria.
- Prioritize backlog by business value / risk.
- Clarify requirements for the team.
- Make decisions quickly to unblock the team.

Anti-patterns:

- PO unavailable or overruled by managers.
- PO micromanaging the team or failing to prioritize.

Scrum Master

- Servant-leader for the Scrum Team.
- Facilitates Scrum ceremonies.
- Removes impediments and coaches team and organization on Agile/Scrum.
- Protects the team from external distractions.

Responsibilities:

- Run Sprint Planning, Daily Scrum, Sprint Review, Retrospective logistics.
- Coach on estimation, definition of done, Scrum rules.
- Help with continuous improvement.

Anti-patterns:

- Scrum Master acts as project manager assigning tasks.

- Scrum Master absent or too directive.

Development Team (or Developers)

- Cross-functional professionals who deliver increments.
- Self-organizing — decide how to accomplish work.
- Typically 3–11 people (7 ± 2 recommended).

Responsibilities:

- Create the Sprint Backlog and deliver increment meeting the Definition of Done (DoD).
- Collaborate, estimate, test and integrate.

Anti-patterns:

- Specialized silos within team that resist cross-functionality.
- Team members treated as interchangeable resources.

Scrum Framework — Full Example

✓ Scrum Roles

Product Owner (PO)

- Priya (PO) gathers requirements from stakeholders.
- Maintains the **Product Backlog**.
- Defines acceptance criteria.

Scrum Master

- John (SM) facilitates ceremonies, removes blockers (e.g., delay from API vendor).

Development Team

- 6 developers (Android/iOS developers + backend + tester + UX designer).
- Cross-functional and self-organizing.

3.2 Scrum ceremonies (events)

Sprint

- Timeboxed iteration, typically 1–4 weeks (2 weeks common).
- Fixed length during which a usable increment must be delivered.
- Sprint includes planning, daily scrums, development, review and retrospective.

Sprint Planning

Purpose: Decide what will be delivered in the sprint and how.

Inputs: Product Backlog, team velocity/capacity, Definition of Done.

Outputs: Sprint Goal, Sprint Backlog (set of selected Product Backlog Items + tasks).

Typical structure:

- Part 1: "What" — PO presents prioritized backlog; the team selects items aligning to Sprint Goal.
- Part 2: "How" — Team plans how to deliver items, decomposes into tasks.

Timebox: ~2 hours per week of sprint length (e.g., 4 hours for a 2-week sprint).

Daily Scrum (Daily Stand-up)

Purpose: Synchronize daily work, inspect progress toward Sprint Goal, surface impediments.

Format: 15 minutes, same time/location. Often each member answers:

- What did I do since yesterday?
- What will I do today?
- Any blockers?

Best practice: Focus on achieving Sprint Goal and coordination (not status reporting to PO).

Sprint Review

Purpose: Inspect the increment with stakeholders and adapt the Product Backlog.

Format: Demo working software, gather feedback, discuss marketplace and roadmap changes.

Participants: Scrum Team + stakeholders.

Outcome: Revised Product Backlog priorities; potential release decisions.

Sprint Retrospective

Purpose: Reflect on the sprint and create plan for improvement.

Format: Identify what went well, what didn't, experiments/practices to try next sprint.

Output: Concrete improvement actions for next sprint.

Timebox: ~45 minutes per week of sprint length (e.g., 90 minutes for a 2-week sprint).

Scrum Ceremonies with Example Events

Sprint Duration: 2 weeks

1. Sprint Planning

- PO presents top stories:
 - User Login
 - Browse Restaurants
 - Add Items to Cart
 - Checkout & Payment
- Team selects stories worth **20 story points** (based on velocity).

Sprint Goal:

| Build the basic ordering flow so users can place an order.

2. Daily Scrum

Typical stand-up update:

- Dev A: "Yesterday built login UI, today integrating API. Blocked by slow response from auth server."
- SM notes the issue and coordinates with DevOps to fix the server.

3. Sprint Review

Team demos:

- Login working
- Restaurant list visible
- Basic cart functioning

Stakeholders give feedback:

- Ask to add "Most Popular Near You" section earlier.
→ PO reorders backlog.

4. Sprint Retrospective

Team identifies:

- Good: Fast collaboration, automated build pipeline.
- Bad: API documentation unclear → slowed development.
- Improvement: Schedule a weekly sync with API vendor.

3.3 Scrum artifacts

Product Backlog

- Ordered list of all work required for product (features, bug fixes, tech debt, research).
- Owned by Product Owner.
- Items typically written as user stories, epics, bugs, chores.
- Continuously refined (Backlog Refinement/Grooming).

Backlog item attributes:

- Title

- Description (user story)
- Acceptance criteria
- Estimate (story points)
- Priority
- Dependencies
- Size (epic vs story)

Sprint Backlog

- Subset of Product Backlog items selected for the sprint + plan to deliver them (tasks).
- Owned by Development Team.
- Emergent and updated daily.

Increment

- Sum of all completed Product Backlog items during a sprint, meeting Definition of Done.
- Must be potentially releasable.

Definition of Done (DoD)

- Team's checklist determining when an increment is "done" (e.g., code review, tests passed, documentation updated, deployment pipeline green).
- DoD should be transparent and enforced—enables consistent quality.

Scrum Artifacts in the Example

Product Backlog (sample items)

Priority	Item	Story Points
1	User Login	5
2	Browse Restaurants	8

Priority	Item	Story Points
3	Add to Cart	5
4	Checkout & Payment	13
5	Live Order Tracking	20
6	Past Order History	13

Sprint Backlog for Sprint 1

- User Login (5)
- Browse Restaurants (8)
- Add to Cart (5)

Total = 18 story points

Increment

At the end of Sprint 1, the increment includes:

- Working login module
- Restaurant browsing
- Basic cart flow

All meeting the **Definition of Done**:

- Code reviewed
 - Tests passed
 - Integrated in build pipeline
 - Demo-ready
-

4. Agile Estimation and Planning

Estimating and planning in Agile are about forecasting, not exact commitments. Emphasis on relative sizing, team velocity, and regular re-planning.

4.1 Story Points — concept

- **Story Points** are a relative measure of effort for a user story (complexity + risk + uncertainty + volume).
- Not time-based: compare stories to each other (e.g., Story A = 2 points, Story B = 5 points).
- Common scales: Fibonacci-ish (1, 2, 3, 5, 8, 13, 21), powers of two, T-shirt sizes (XS, S, M...).

What story points capture:

- Complexity (technical difficulty).
- Amount of work.
- Uncertainty and risk.

Advantages vs hours:

- Encourage relative sizing and team calibration.
- Avoid false precision of time estimates.
- History of velocity helps forecasting.

4.2 Planning Poker (estimation technique)

Planning Poker — steps:

1. Each story is discussed briefly by team and PO.
2. Team members privately choose a card (story point value) representing their estimate.
3. Everyone reveals simultaneously.
4. If estimates differ, high and low explain reasoning.
5. Repeat until consensus reached (or use average/modified consensus).

Benefits:

- Democratic, encourages discussion and knowledge sharing.
- Exposes assumptions and risks.

Common practices:

- Use a moderator (often Scrum Master).
- Limit discussion time per story for efficiency.
- Combine with backlog refinement sessions.

4.3 Estimation best practices

- Estimate as a team (not single person).
- Use reference stories anchored to known points (e.g., a specific story is a "5").
- Re-estimate after large changes; keep estimates stable for comparability.
- Avoid mixing story points and time in the same metric.

4.4 Release Planning

Purpose: Forecast when a set of features will be delivered to stakeholders/customers.

Inputs:

- Product Backlog (prioritized & estimated).
- Team velocity (historical average of completed story points per sprint).
- Team capacity (available sprint hours, vacations, holidays).

Method:

- Calculate total story points for target release.
- Use velocity to compute number of sprints:

Example: if total = 120 story points, avg velocity = 20 points/sprint → $120 \div 20 = 6$ sprints.

(Note: always add buffer for uncertainty — e.g., 10–30%.)

Communicate: Release plan is a forecast, not a commitment. Update after each sprint.

4.5 Iteration (Sprint) Planning

Purpose: Commit to what the team will deliver in the next sprint.

Inputs: Sprint Goal, highest-priority Product Backlog items, team capacity, DoD.

Steps:

- PO presents top items and sprint goal.
- Team selects items it can realistically complete based on velocity and capacity.
- Decompose stories into tasks (optional time estimates) and identify dependencies.
- Finalize Sprint Backlog & Sprint Goal.

Capacity considerations:

- Consider holidays, planned time off, non-project work (meetings/support).
- Use team capacity (person-days) rather than headcount.

Example capacity calc:

- Team of 5, sprint length 10 working days = 50 person-days.
- Subtract planned overhead: 10% meetings → $50 \times 0.9 = 45$ effective person-days.
- Convert to story points using historical productivity ratio if needed.

Agile Estimation & Planning — Real Example

Planning Poker Estimation

Team estimates “Live Location Tracking” story:

- Dev A picks: 13
- Dev B picks: 20
- Tester picks: 20

Discussion reveals complex GPS + real-time socket backend → final estimate = **20 points**.

Release Planning Example

Total backlog = 120 story points

Velocity = 20 points/sprint

Estimate: **$120 \div 20 = 6$ sprints**

Stakeholders understand this is a **forecast**, not a fixed commitment.

Sprint Planning — Capacity Check

- Team of 6 → approx. 60 person-days
 - After meetings & holidays → effective = 48 days
- PO selects stories worth ~20 points to match historical velocity.
-

5. Agile User Stories

User stories are the primary way of capturing requirements in Agile.

5.1 User story structure

Typical template:

| As a [user role], I want [capability], so that [benefit/value].

Examples:

- As a **customer**, I want **to reset my password via email**, so that **I can regain access if I forget it**.
- As an **admin**, I want **to export user reports in CSV**, so that **I can analyze activity in Excel**.

5.2 INVEST criteria for good stories

A good user story often follows INVEST:

- **Independent** — can be developed independently if possible.
- **Negotiable** — not a contract, open to discussion.
- **Valuable** — delivers value to user/customer.
- **Estimable** — small enough to estimate.
- **Small** — fits within a sprint.
- **Testable** — acceptance criteria must be clear so story can be verified.

5.3 Epics and splitting stories

- **Epic**: Large feature that should be broken into multiple stories.
- **Splitting strategies**:
 - By user role or personal.
 - By workflow step (login → verify → use).
 - By business rule (basic vs advanced).
 - By CRUD operations (Create, Read, Update, Delete).
 - By happy path vs edge cases.
 - By UI vs backend.

5.4 Acceptance criteria (AC)

Acceptance Criteria define the conditions that must be met for a story to be accepted as done.

Formats:

- **Given/When/Then** (BDD style):
 - Given [precondition], When [action], Then [expected result].
- **Checklist**:
 - ☐ AC 1: ...
 - ☐ AC 2: ...

Examples:

User story: *As a user, I want to reset my password via email so I can regain access if I forget it.*

Acceptance criteria:

- Given an existing user on “Forgot password” page, when they submit their registered email, then the system sends a password reset email containing a unique token valid for 1 hour.
- Given user clicks the token link within 1 hour, when they enter a new password that meets complexity rules, then the password is updated and a confirmation email is sent.
- Given token expired, when user uses link, then show an expiration message and option to request a new reset.

Acceptance Test Examples:

- Functional tests: unit/integration/UI tests validating ACs.
- Manual test steps mapped to ACs.

5.5 Definition of Ready (DoR)

Optional checklist the team uses to ensure a story is ready for sprint planning (e.g., clear description, acceptance criteria, estimate, no blocking dependencies, design in place).

User Story Example (Complete)

User Story

As a customer, I want to track my food delivery in real-time so I know exactly when it will arrive.

Acceptance Criteria (AC) — Using Given/When/Then

1. **Given** the order is accepted, **when** I open “Track My Order”, **then** I see the delivery partner’s live location.

2. **Given** the order is out for delivery, **when** I refresh tracking, **then** the map updates within 3 seconds.
3. **Given** GPS signal is weak, **then** an error message "Location unavailable" is shown.

Definition of Ready (DoR) Check

- AC provided ✓
 - Design mock-ups done ✓
 - API specs confirmed ✓
 - No dependencies ✓
-

6. Agile Metrics and Reporting

Measurements help teams inspect and adapt. Use metrics carefully—focus on improving outcomes, not gaming numbers.

6.1 Key metrics

Velocity

- Definition: Average number of story points the team completes per sprint.
- Calculation: Sum of story points completed in each sprint, averaged over last N sprints (commonly 3–6 sprints).
- Use: Forecasting, capacity planning.
- Caution: Do not compare velocities across teams (different estimation scales).

Example:

Sprints with completed points: 18, 22, 20, 24, 16.

Calculate average velocity over these 5 sprints:

Step-by-step:

1. Sum = 18 + 22 + 20 + 24 + 16.

- $18 + 22 = 40$.
- $40 + 20 = 60$.
- $60 + 24 = 84$.
- $84 + 16 = 100$.

2. Average = $100 \div 5 = 20$.

So velocity = **20 story points/sprint**.

Burndown Chart

- Shows remaining work (story points or tasks) versus time during a sprint.
- X-axis: time (days); Y-axis: remaining work.
- Ideal burndown: smooth decline to zero at sprint end.
- Use: Quick view of progress, scope changes, and risks.
- Interpretations:
 - Flat line: no work completed → risk.
 - Upward bumps: scope added or estimates changed.
 - Rapid decline then plateau: early completion then obstacles.

Burnup Chart

- Alternative: shows work completed and total scope line. Good for visualizing scope creep.
- Two lines: total scope (may change) and completed work. When lines meet, release done.

Cumulative Flow Diagram (CFD)

- Shows work in different states (e.g., Backlog → In Progress → Testing → Done) over time.
- Useful to spot bottlenecks: widening bands indicate accumulation in a state.

Escaped Defects / Defect Density

- Number of defects found in production per release / per lines of code / per function point.
- Use to gauge quality and prioritize technical improvements.

Lead Time and Cycle Time

- **Lead time:** Time from request (story creation) to delivery.
- **Cycle time:** Time from when team starts work on an item to its completion.
- Shorter times indicate faster feedback and throughput.

Throughput

- Number of items (stories, tickets) completed in a period. Useful in Kanban contexts.

Work in Progress (WIP)

- Number of items in active states. Limiting WIP reduces multitasking and improves flow.

6.2 Reporting progress in Agile projects

Regular reports and artifacts

- **Sprint burndown:** daily update during sprint.
- **Release burnup:** track progress toward release scope.
- **Velocity chart:** trend of velocity over sprints.
- **CFD:** monitor state distribution and bottlenecks.
- **Sprint Review outcomes:** showcase working increment + feedback captured in backlog.
- **Retrospective actions:** tracked to measure continuous improvement.

Stakeholder communication

- Provide short demos at Sprint Review — show working features, not slides.
- Share release forecasts (based on velocity) and risk/assumption logs.

- Provide high-level dashboards for executives (release % complete, projected release date, key risks).
- Keep metrics focused on value delivered (features shipped, customer satisfaction), not vanity metrics.

6.3 How to use metrics responsibly

- Use metrics to **inspect and adapt**, not to punish the team.
- Avoid local optimization (e.g., forcing faster velocity by inflating story points).
- Combine quantitative metrics with qualitative feedback (customer satisfaction, NPS).
- Revisit and calibrate estimation and metrics periodically.

Agile Metrics & Reporting (Example Data)

Velocity (Example)

Sprint	Completed Story Points
1	17
2	20
3	19
4	20

➡ Average = **19 SP**

Used for forecasting future sprints.

Burndown Chart Example (Sprint 2)

- Day 1 → 22 SP remaining
- Day 5 → 14 SP remaining
- Day 10 → 0 SP

Blocked mid-sprint by payment API issue → visible in flat line on chart.

Burnup Example

Shows:

- Total work = 120 SP
- Completed = 78 SP after Sprint 4
- Scope increased from 120 → 130 SP when stakeholders added "Promo Coupons"

Burnup makes **scope change visible**.

Cumulative Flow Diagram

"Testing" column widened → testing bottleneck found.

Solution: start QA earlier + add automation tests.

7. Practical Examples & Templates

7.1 Example user story with AC and tests

Story: As an **e-commerce customer**, I want **to add items to a wishlist**, so that I **can save items for later purchase**.

Acceptance Criteria (Given/When/Then):

1. Given I am a logged-in customer, when I click "Add to wishlist" on a product page, then the product is added to my wishlist and a confirmation toast is shown.
2. Given an item is in my wishlist, when I view my wishlist, then I should see the item with product name, price, and link to product page.
3. Given I am not logged in, when I attempt to add an item to wishlist, then I am prompted to log in or create an account.

Tests:

- Unit test for wishlist add/remove service.
- Integration test for wishlist persistence across sessions.
- Manual UI test: add item, confirm toast and item in wishlist.

7.2 Sample Sprint Planning template

- Sprint Goal:
- Sprint Length:
- Team capacity (person-days):
- Planned Sprint Backlog (stories with story points):
 - Story 1 — 8 points — Owner: Dev A
 - Story 2 — 5 points — Owner: Dev B
 - ...
- Tasks per story (with rough hours if desired):
- Risks / Blockers:
- Definition of Done snapshot:
- Acceptance criteria still pending?

7.3 Definition of Done (example checklist)

- Code compiles and passes unit tests.
 - Integration tests pass.
 - Peer code review completed and approved.
 - Functionality tested on staging environment.
 - Documentation updated (user docs, API docs).
 - Performance criteria met.
 - No critical or high-severity defects open.
 - Release notes prepared.
-

8. Common Pitfalls & Anti-Patterns — and how to avoid them

Pitfalls

- **Misusing Velocity:** Comparing across teams or using as a productivity metric leads to bad behavior.
 - *Fix:* Use velocity only for team forecasting; combine with qualitative insights.
- **Incomplete stories in sprint:** Frequent unfinished stories.
 - *Fix:* Better slicing of stories, realistic sprint planning, look at root causes in retrospectives.
- **Over-commitment:** Team takes more work than capacity allows.
 - *Fix:* Use historical velocity and calculate capacity accurately.
- **Micromanagement by Product Owner or managers:**
 - *Fix:* PO focuses on “what” and “why”, team decides “how”; Scrum Master protects team.
- **Skipping retrospectives or not acting on outcomes:**
 - *Fix:* Make retrospectives engaging and turn insights into concrete experiments; assign owners.
- **Too much documentation or too little:**
 - *Fix:* Aim for “just enough” documentation—valuable, up-to-date, and linked to working software.
- **Ignoring technical debt:**
 - *Fix:* Include tech debt stories in backlog; allot time (e.g., 10–20%) for technical improvements.

Anti-patterns to watch

- “Water-Scrum-Fall”: only scrum in middle, upstream/downstream waterfall processes remain.
 - “Cargo cult Agile”: doing ceremonies mechanically without embracing principles.
 - Using story points as a performance KPI or to reward teams.
-

9. Scaling Agile

When multiple teams work on the same product, consider a scaling approach (examples: SAFe, LeSS, Nexus). Key concepts:

- Align on product vision and roadmap.
- Coordinate cross-team dependencies (e.g., via an ART or integration sprint).
- Shared backlog grooming and joint planning (PI Planning in SAFe).
- Keep teams as independent as possible and minimize cross-team blocking.